If you were running this on hundreds of thousands of rows, you may wish to update a batch of, say, 10,000 rows at a time. Working on batches of rows will be covered later on. For now, consider how the Direct Updates technique can be as simple as a single statement for all rows.

Compared with Table Copying, you don't need to worry about Foreign Key Constraints, Sequence modifications, or Index creations.

However, to restore optimal performance, when using Direct Updates you'll want to perform some post-update database maintenance.

What does that look like?

Performing Database Maintenance

Because you've updated every row, a new row version has been created and taken over. Remember that row versions are immutable in PostgreSQL. The former version is now a "dead" row. The space consumed by the dead rows can be reclaimed. Normally the *Autovacuum* process helps keep things tidy and runs automatically for you. Autovacuum is a background scheduler system that runs a `VACUUM` operation for each of your tables.

For manual operations like Direct Updates where you're modifying many rows, you may wish to immediately perform an equivalent operation to restore optimal performance. As the PostgreSQL operator you're able to run a manual `VACUUM` on a table at any time, and this is safe to do.

To run a manual `VACUUM`, from `psql` run this statement:

```
VACUUM (ANALYZE, VERBOSE) users;
```

This command adds the `ANALYZE` and `VERBOSE` arguments.

With `ANALYZE` you're also updating the statistics that are collected for the table; PostgreSQL keeps track of some of the properties of the data in the table when the `ANALYZE` command runs.

The `VERBOSE` option adds much more output about what's happening, listing the number of pages being scanned, live rows, and dead rows.

Another important maintenance operation after a big update is to refresh Indexes for the table. This operation is called *Reindexing* and the Index is actually rebuilt instead of being updated directly. To rebuild indexes you'll use the `REINDEX` command.

For PostgreSQL version 12, the `REINDEX` command gained a new option called `CONCURRENTLY`. This is very significant because it makes it possible to rebuild